INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

Subir Chowdhury School of Quality and Reliability

TOPIC: Software Fault Prediction using Artificial Neural Network

Spring (2025 – 2026)

Submitted by-

SUTAMA SARKAR (25RE61R07)

Introduction

1.1 Software Fault Prediction (SFP) is a critical activity in the software development life cycle. The goal is to predict the cumulative number of faults (m (t)) at a given testing time (t).Artificial Neural Networks (ANN) provide a "black-box" approach that can model complex.

1.2 Problem Statement: The goal of this project is to develop an ANN-based model where:

- **Input (T):** Testing Time (Interval index).
- **Output (m (T)):** Cumulative number of faults detected up to time T.
- **Objective:** To train the network to learn the reliability growth curve and predict future fault occurrences accurately.

1.3 Dataset Selection: The dataset was sourced from a software testing repository (Pharmdata) containing 111 observations.

- **Input Vector:** Time intervals from 1 to 111.
- **Target Vector:** Cumulative faults derived from the raw fault data (starting from 5 and reaching a total of 481).

Time	Faults
1	5
2	5
3	5
4	5
5	6
6	8
7	2
8	7
9	4
10	2
11	31
12	4
13	24
14	49
15	14
16	12
17	8
18	9
19	4
20	7
21	6
22	9
23	4
24	4
25	2
26	4
27	3
28	9
29	2
30	5
31	4
32	1
33	4
34	3
35	6
36	13
37	19
38	15
39	7
40	15
41	21
42	8
43	6
44	20
45	10
46	3
47	3
48	8
49	5
50	1
51	2
52	2
53	2
54	7
55	2
56	0
57	2
58	3
59	2
60	7
61	3
62	0
63	1
64	0
65	1
66	0
67	0
68	1
69	1
70	0
71	0
72	1
73	1
74	0
75	0

76	0
77	1
78	2
79	0
80	1
81	0
82	0
83	0
84	0
85	0
86	0
87	2
88	0
89	0
90	0

91	0
92	0
93	0
94	0
95	0
96	1
97	0
98	0
99	0
100	1
101	0
102	0
103	1
104	0
105	0

106	1
107	0
108	0
109	1
110	0
111	1

1.4 Steps for Software Fault Prediction using MATLAB ANN Fitting

Step 1: Data Initialization: The time intervals and cumulative fault counts are entered as vectors in the Command Window.

- **Command:** T = [1, 2, ...111]; F = [5, 10, ...];

Step 2: Launching the Neural Net Fitting App: The Neural Network Start wizard is invoked to handle the regression task.

- **Command:** Directly open neural network fitting from “APPS” in online matlab.
- **Input/Target Selection:** Assign the time vector (T) as **Predictors** and the fault vector (F) as **Targets**.

Step 3: Network Configuration: Before training, the data distribution and architecture are defined

- **Data Splitting:** The dataset is divided into:
  - **Training (80%):** Used by the network to adjust weights.
  - **Validation (10%):** Used to stop training before overfitting occurs.
  - **Testing (10%):** Used as an independent measure of network performance.
- **Neuron Selection:** The hidden layer is set to **10 neurons** to provide the necessary complexity to map the non-linear reliability growth curve.

Step 4: Model Training: The network is trained by using algorithm.

Step 5: Performance Evaluation & Plotting: After training, five key plots are generated to verify the model's accuracy-

Performance Plot, Training State Plot, Error Histogram Plot, Regression Plot, Function Fit

Step 6: Exporting Results

Step 7: Model Validation and Future Prediction: After training the model, it is essential to verify its accuracy against known values and use it to forecast future failures. This was performed using the MATLAB Command Window.

7.1 Validation at T = 111 days

To check the consistency of the ANN model, a prediction was made for the final data point in the dataset (T = 111).

- **Command:** new_timedata = [111];

  predict_f = resultdata.Network(new_timedata);

- **Result:** The ANN predicted **480.5291** faults.
- **Comparison:** This value is extremely close to the analytical value (Goel Okmoto) of **480.9984**, showing that the network has accurately "learned" the endpoint of the testing phase.

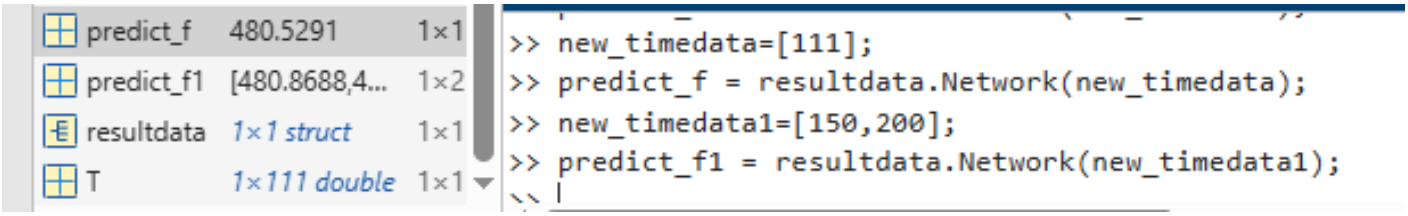
7.2 Future Fault Prediction

One of the primary uses of this ANN model is to predict the total number of faults if testing were to continue beyond the current duration. Predictions were made for (T = 150) and (T = 200)

- **Command:** new_timedata1 = [150, 200];

  predict_f1 = resultdata.Network(new_timedata1);

- **Result:** The predicted total faults at T = 200 days is **480.8688**.



1.5 Results and Outcomes: Training = 80%, Validation and Testing = 10% and 10%
Layer Size = 10

Training Results

Training finished: Met validation criterion

Training Progress

Unit	Initial Value	Stopped Value	Target Value
Epoch	0	134	1000
Elapsed Time	-	00:00:00	-
Performance	3.75e+05	6.9	0
Gradient	6.07e+05	3.53	1e-07
Mu	0.001	0.001	1e+10
Validation Checks	0	6	6

Fig 1.5.1: Training Result

➢ **Inferences:**

- The neural network **successfully learned** from the data
- Training stopped early due to **validation-based stopping**, which is good practice
- The model achieved a **significant reduction in error**, indicating good performance
- The stopping condition suggests the model reached its **optimal generalization point**

Train a neural network to map predictors to continuous responses.

Data

Predictors: T - [1x111 double]

Responses: F - [1x111 double]

T: double array of 111 observations with 1 features.

F: double array of 111 observations with 1 features.

Algorithm

Data division: Random

Training algorithm: Levenberg-Marquardt

Performance: Mean squared error

Training Results

Training start time: 26-Mar-2026 13:53:44

Layer size: 10

	Observations	MSE	R
Training	89	6.9995	0.9998
Validation	11	0.7366	0.9995
Test	11	2.7081	0.9995

Fig: Training Outcomes

➢ **Inferences:**

- The low mean squared error values indicate that the model predictions are **very close to the actual target values**.
- Consistency across all datasets confirms that the model has **good generalization ability** and is not overfitting.

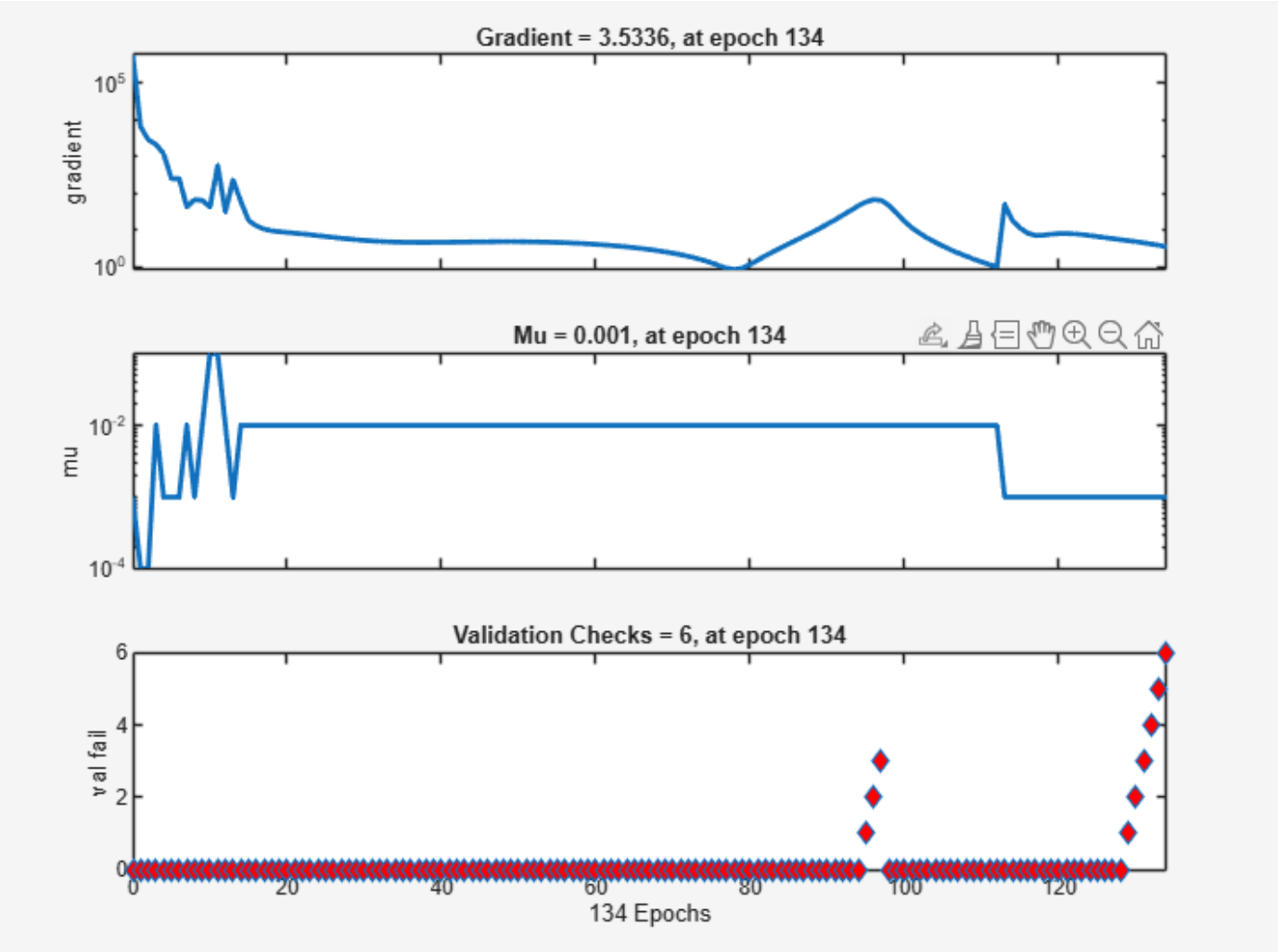


Fig: Time state plot

➢ **Inferences:**

- The network shows **fast initial learning**, followed by stable convergence
- The training process is **well-controlled and efficient** due to adaptive μ
- Early stopping based on validation checks ensures the model does **not overfit**
- Final epoch (134) represents the **optimal stopping point**

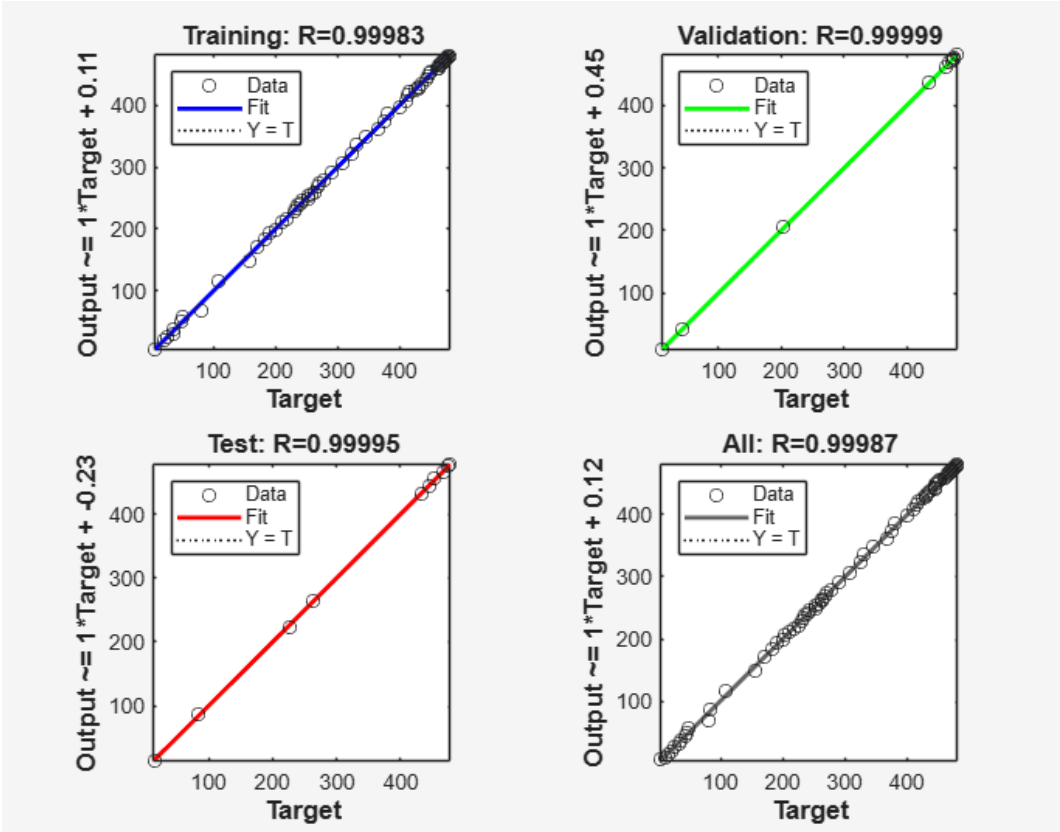


Fig: Regression Plot

- **Inferences:**

- The neural network model demonstrates **near-perfect prediction capability**
- The strong alignment between outputs and targets confirms that the model has successfully captured the underlying relationship in the data
- Consistency across all datasets indicates a **robust and reliable model**

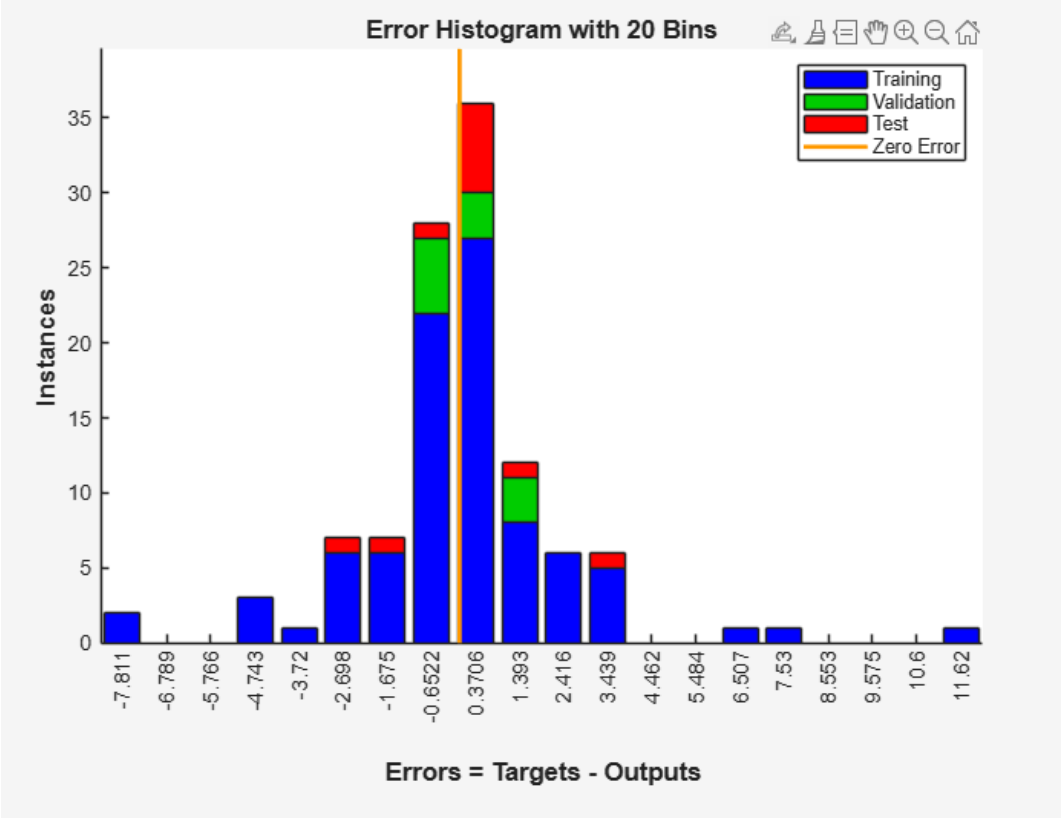


Fig: Error Histogram Plot

- **Inferences:**

- The neural network produces **small and well-distributed errors**
- Predictions are **unbiased and consistent**
- The concentration of errors near zero confirms **high model r**

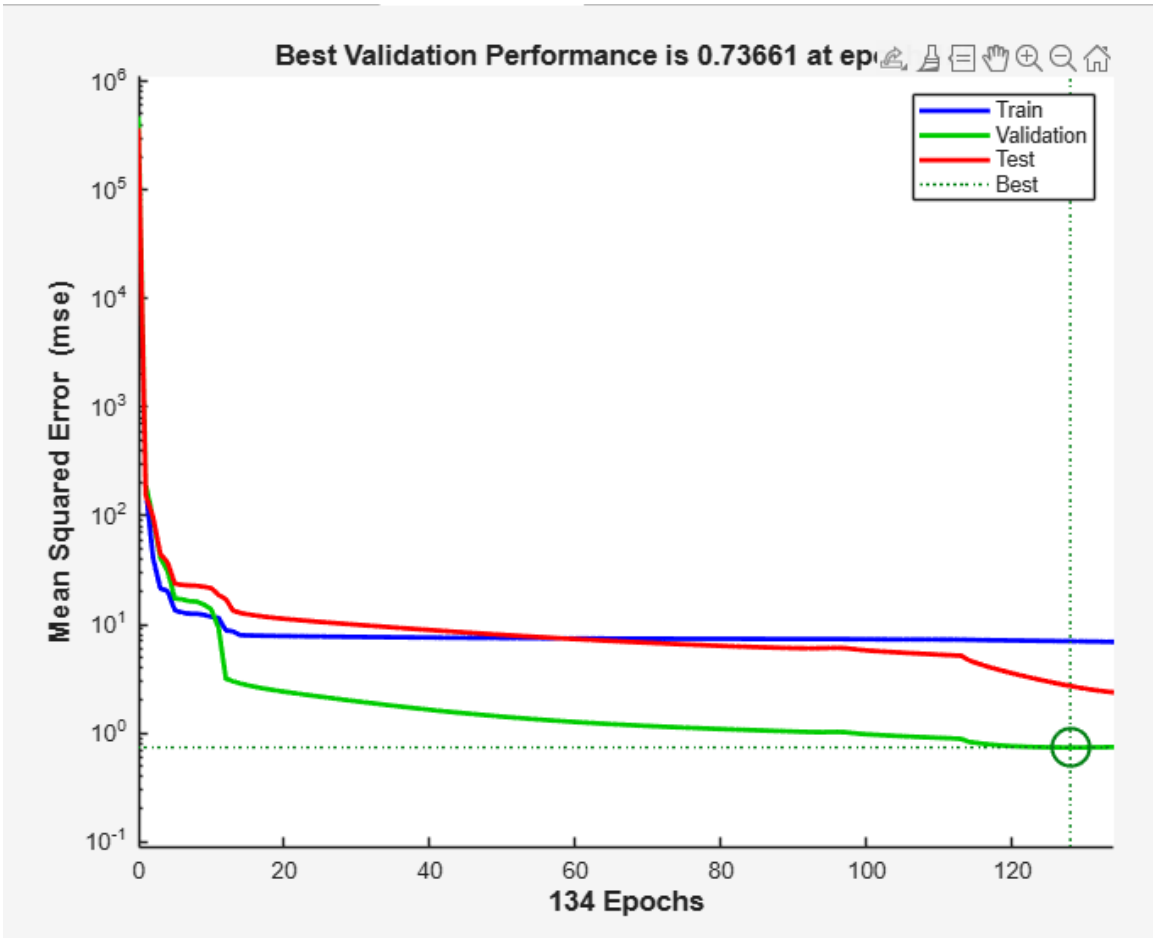
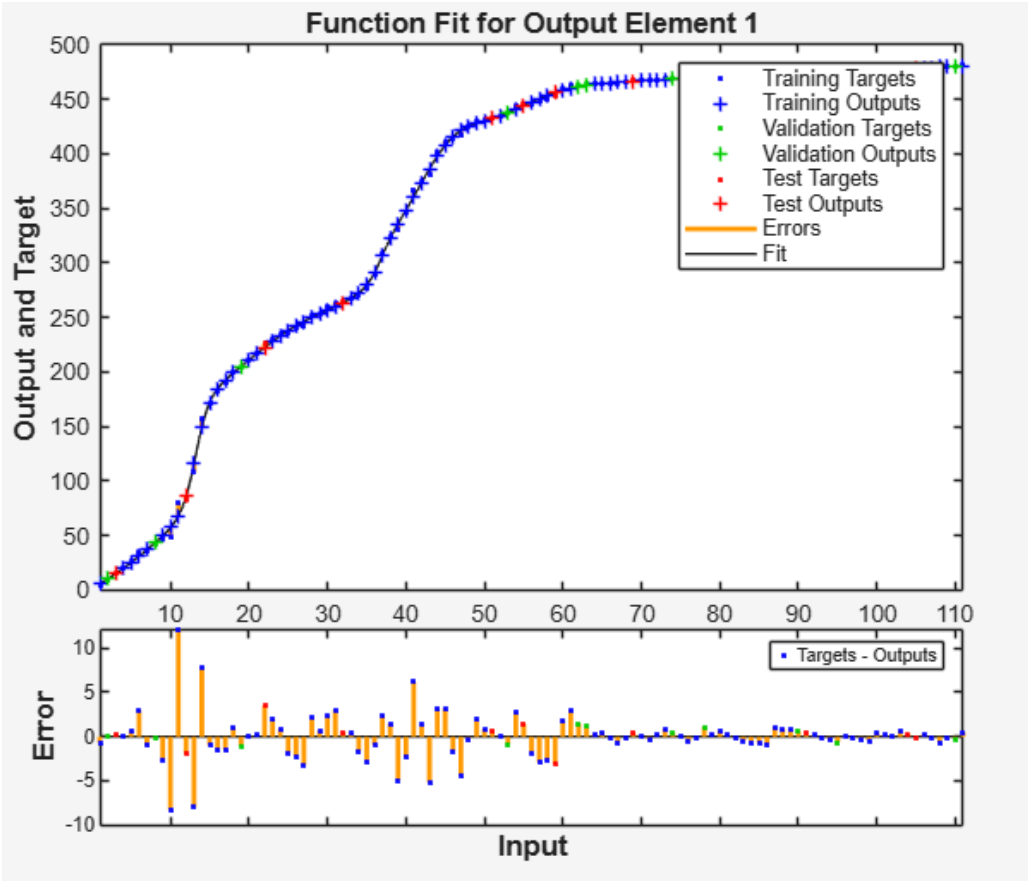


Fig: Performance Plot

➢ **Inferences:**

- The neural network shows **fast convergence**, **low error**, and **stable performance**
- The close alignment of training, validation, and test errors confirms a **well-trained and reliable model**
- Early stopping ensures the model is **optimized without overfitting**



Training - 80% Validation and Testing - 10% and 10% Layer Size = 10

Fig: Fit plot

➢ **Inferences:**

- The neural network provides an **excellent fit** to the data
- It accurately models the nonlinear relationship between input and output
- Errors are minimal and randomly distributed, confirming **robust and unbiased predictions**